

RAG • LLM • VECTOR SEARCH

AI Medical Chatbot using RAG

MediBot

A modular three-phase pipeline for document-grounded question answering

Technical Presentation

LangChain • HuggingFace • Mistral • FAISS • Streamlit • Python

Project Overview

Source content retained, with supporting RAG context added

What is MediBot?

A modern AI chatbot for documents that uses Retrieval Augmented Generation (RAG) to connect an LLM with information stored in a vector database.

Core idea

Instead of relying only on model memory, MediBot retrieves relevant document chunks and uses them as context for the answer.

Project structure

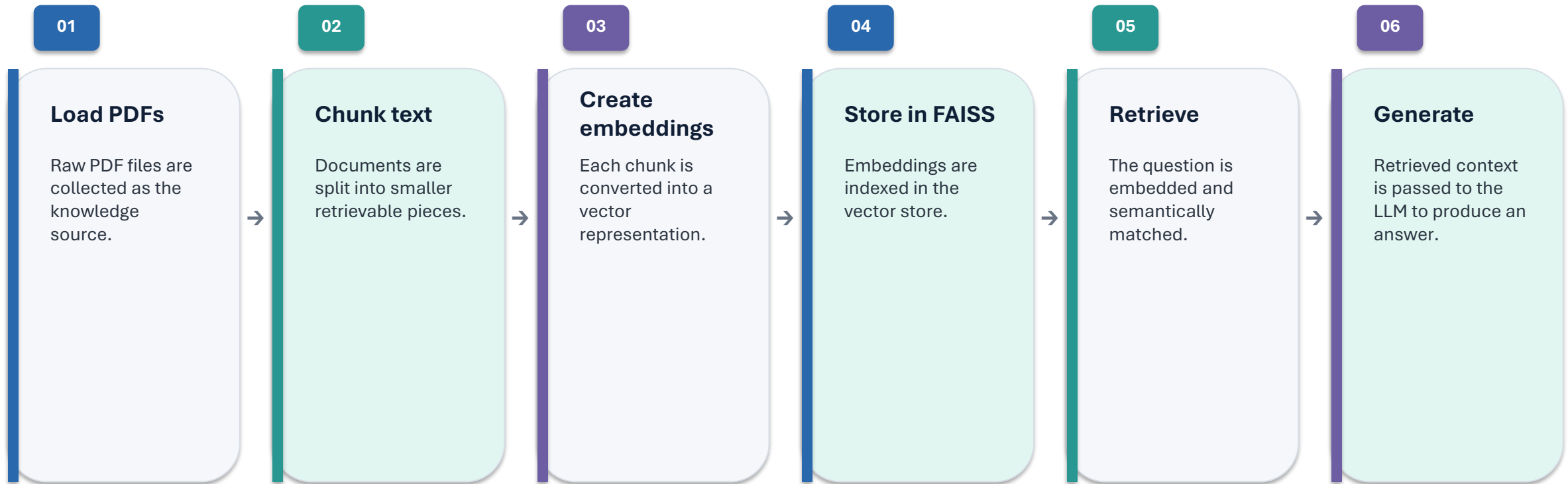
The implementation is organized into three phases: vector memory setup, LLM connection, and Streamlit UI.

Why RAG fits this project

- Ground responses in the project's document collection
- Use semantic similarity instead of simple keyword matching
- Keep retrieval and generation as separate, modular stages
- Make the pipeline easier to extend with additional documents

End-to-End RAG Workflow

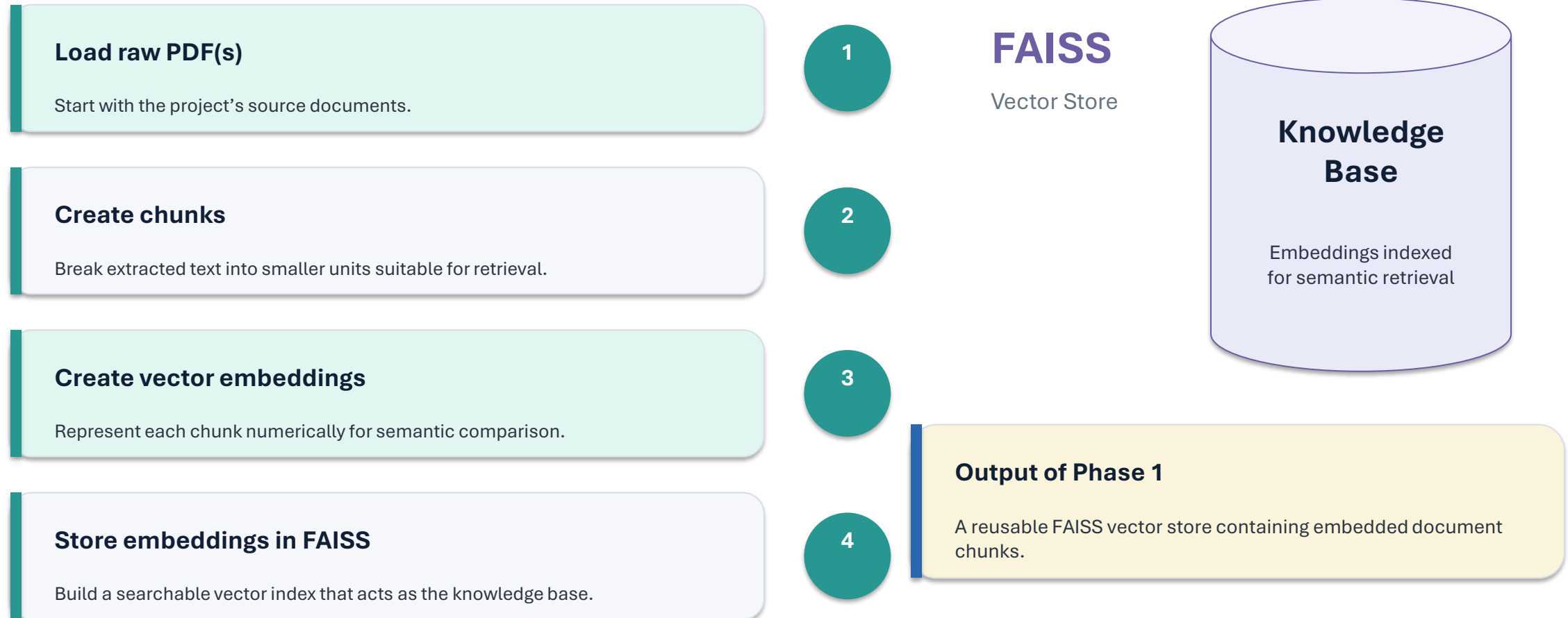
High-level flow from source documents to a user-facing answer



Key principle: retrieval supplies relevant context; generation turns that context into a natural-language response.

Phase 1 — Setup Memory for the LLM

Vector database preparation



Phase 2 — Connect Memory with the LLM

Retrieval + generation

1. Setup LLM

Mistral with HuggingFace is used as the language model.

2. Connect to FAISS

The LLM workflow is connected to the vector store for retrieval.

3. Create chain

A chain coordinates the question, retrieval, context, and response generation.

Question → Question Embedding → Semantic Search → Relevant Context → Mistral LLM

Why semantic retrieval?

The query and document chunks are compared in vector space, helping retrieve content by meaning rather than exact wording.

Phase 3 — Streamlit Chatbot UI

Simple interface for asking questions and viewing generated answers

User interaction

The user enters a question in the chatbot interface and receives an answer.

Cached vector store

FAISS can be loaded into cache so the application can reuse the prepared vector store.

RAG response

The application retrieves relevant context and passes it into the generation flow.

Example interaction

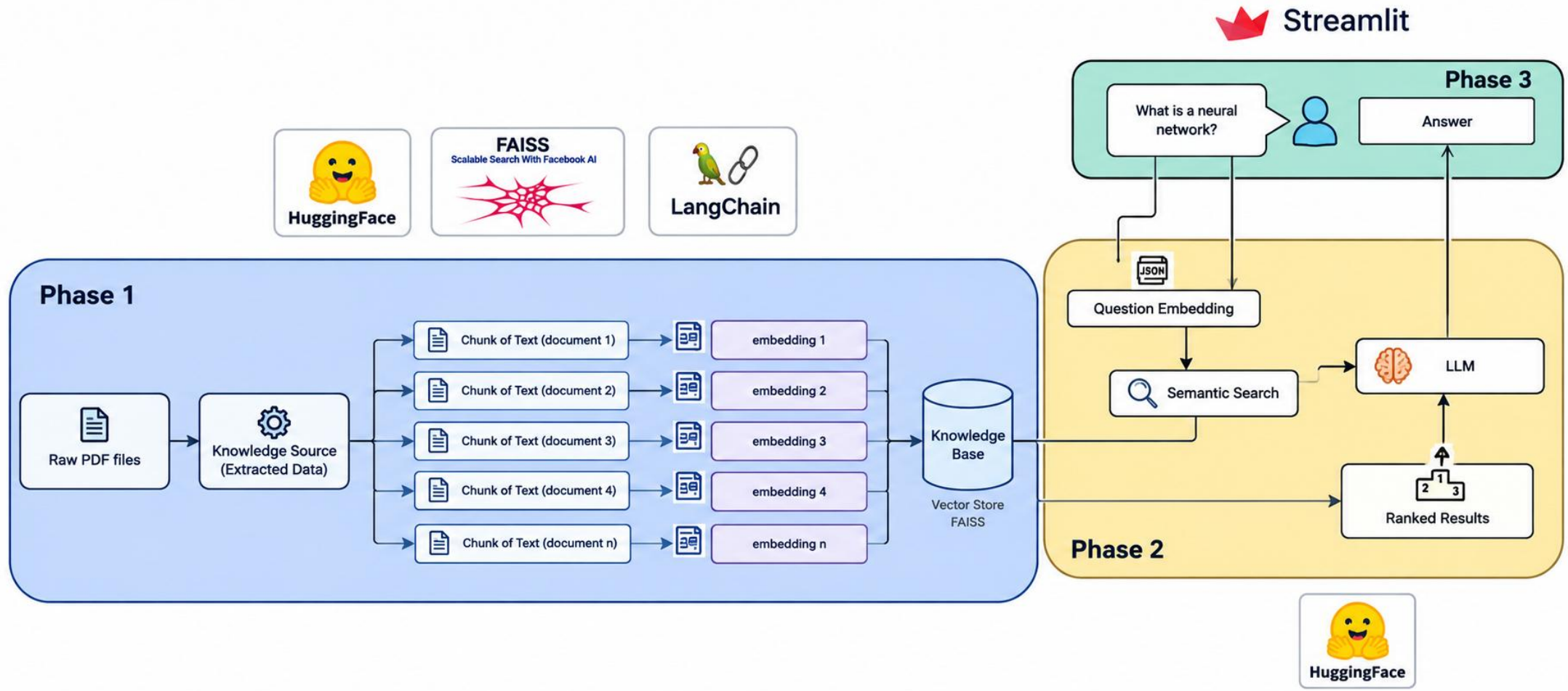
User: What is a neural network?

MediBot: Answer generated using retrieved context.

Streamlit provides the presentation layer; RAG provides the knowledge-grounded workflow.

Technical Architecture

Original architecture from the supplied project deck



Tools & Technologies

Technology stack retained from the source, with role-based explanation

LangChain

AI framework for LLM applications

HuggingFace

ML / AI hub

Mistral

LLM model used in the pipeline

FAISS

Vector database / similarity index

Streamlit

Chatbot UI

Python

Programming language

VS Code

IDE / development environment

Development workflow

Python + VS Code bring the RAG components together into the application.

RAG Components Explained

Supporting concepts that make the architecture easier to understand

Chunking

Splits long documents into retrievable units.
Chunk size and overlap affect retrieval quality.

Embeddings

Transforms text into vectors so semantically related content can be compared.

Vector Search

Finds chunks whose embeddings are most similar to the question embedding.

Prompt Context

Retrieved chunks become context for the LLM before response generation.

Generation

The LLM turns the supplied question + retrieved context into natural language.

Caching

Reusing the prepared FAISS store can reduce repeated loading work in the UI.

Improvement Potential / Next Steps

Source roadmap preserved and expanded with practical engineering directions

Original roadmap

- Add authentication in the UI
- Make use of self-upload document functionality
- Add multiple documents and embed them together
- Add unit testing of RAG applications

Related engineering enhancements

- Add retrieval evaluation metrics
- Experiment with chunk size / overlap
- Add source references to retrieved answers
- Add monitoring for retrieval and generation quality

Note: added items are suggested extensions, not claims about the current implementation.

Project Summary

What the audience should remember

Modern AI chatbot for documents

MediBot combines an LLM with a vector-based retrieval layer.

Modular 3-phase project

Memory setup → LLM connection → Streamlit UI.

Core technologies

Streamlit, LangChain, HuggingFace, Mistral, FAISS and Python.

End-to-end RAG pipeline

Documents become embeddings; questions retrieve relevant context; the LLM generates the answer.

Future possibilities

Authentication • Self-upload • Multi-document embedding • RAG testing & evaluation

Thank You

MediBot — AI Medical Chatbot using RAG

Questions & Discussion

PDF

Chunks

FAISS

RAG

LLM

UI

Source deck content preserved • Architecture retained • Supporting RAG concepts added